Machine-Checked Foundations for the Exact-Rational Riccati Certificate: A Lean Companion Note

Adi Oltean*

Version 17 — September 2026

1 What this note is

This note is the verification companion to the paper on exact-rational certification of block-banded trajectory QCQPs [2], and it does three jobs. It documents the Lean 4 formalization behind the machine-checked results cited there: each such citation resolves to the numbered section here that states the corresponding declaration and its hypotheses. It catalogues the executable receipts behind the paper's measured and instance-level results (§6). It records the mechanization of the published Podium technical note's theorems (§7).

The corpus machine-checks classical facts along the paper's trusted path rather than new mathematics: what it supplies is a tolerance-free, mechanically verified chain, not novelty in the individual lemmas.

2 The corpus and its pin

The mathematical acceptance predicates and supporting results are mechanized in nineteen top-level Lean 4 files and `note/PodiumNote.lean`. The exact cited sources are retained in `revision-v17/lean-corpus-v17.zip`; its manifest records each source hash, the toolchain pins and per-file compilation results recorded for that snapshot. Every cited declaration is named explicitly. The snapshot fixes the corpus independently of changes to a repository branch. All twenty files compile without `sorryAx`; the implementation-refinement limits remain those of Section 8.

The manifest pins `leanprover/lean4:v4.33.0` and Mathlib commit `db584cd6d46c92f2 09a44c0f1c829460d327499d`. The retained per-file logs record successful compilation; the manifest separately records the toolchain identity. The snapshot producer, `release/2026-09-08/ check_lean.py` in the paper revision repository, reads the earlier frozen corpus, corrects the `BorderedCriterion.lean` header to name bridge steps (1), (4), (5), and recompiles all twenty files. No Lean declaration or proof changed. The producer uses a local dependency cache; it does not provision dependencies. An independent re-check can run `lake env lean` on the extracted files in the pinned project, as the archive README describes. The retained logs include `#print axioms`

*Starcloud, Inc. `adi@starcloud.com`. AI coding assistants were used for drafting, for the reference implementation and verification scripts, and for adversarial review.

output for the headline declarations. The axiom audit admits only `propext`, `Classical.choice` and `Quot.sound`, and never `sorryAx`.

The retained corpus archive has SHA-256 `6d6e3d018c34337a27f1274f74abda8c1b1c2633 6d02a4e0ed973f668168e695`.

3 The acceptance predicate, both directions

`ldlt_isPSD` (`LDLTSound.lean`): if $M = L^\top \operatorname{diagonal}(d)\, L$ and every pivot $d_i \geq 0$, then M is PSD in the quantifier sense $\forall v,\ 0 \leq v^\top M v$. This is the acceptance direction; it holds at any order and over any linear ordered field, by the expansion $v^\top M v = \sum_i d_i (Lv)_i^2$.

`ldlt_complete` (`SchurStep.lean`): a symmetric PSD matrix of any order m admits weights $d_j \geq 0$ and vectors v_j with $M = \sum_j d_j v_j v_j^\top$. The induction peels one nonnegative-weighted rank-one term per *positive* pivot, and it passes through singular pivots because a zero pivot forces a zero border (`headBordered_border_zero`), contributing no term. This is the converse of `ldlt_isPSD` in content: it yields a $k \times m$ factor with nonnegative weights rather than a square L. The declaration bounds neither k nor the factor's shape; the proof's witness carries one term per positive pivot, so $k \leq m$ there, but that bound is not part of the statement.

`isPSD_iff_ldlt` (same file) packages the two directions as one equivalence: for symmetric M, M is PSD if and only if such a nonnegative-weight decomposition exists. This is the machine-checked form of accept $\Leftrightarrow$ PSD, with *accept* being exactly that existential.

3.1 The bridge to the library-standard notion

`posSemidef_iff_isPSD` (`PosSemidefBridge.lean`): for a rational $n \times n$ matrix M, Mathlib's `Matrix.PosSemidef` M holds exactly when $M^\top = M$ and `IsPSD` M, the quantifier form $\forall v,\ 0 \leq v^\top M v$. Symmetry is derived by the equivalence, not assumed of M. So for symmetric M the quantifier form is the library-standard notion rather than a bespoke one. That bridge is compiled on its own and is not linked inside Lean to the predicate the acceptance lemmas above are stated against; the condition is stated in §8.

4 Certificate soundness and closure

`general_sprocedure_sound` (`GeneralLifting.lean`) takes three hypotheses:

1. the bordered matrix built from curvature block P, half-linear part b and corner c is PSD;
2. every multiplier $\lambda_i \geq 0$;
3. the form identity: for every point, the bordered quadratic form at the lift equals $f_0 - \sum_i \lambda_i f_i - t$.

From these it concludes $t \leq f_0(y)$ at every y with all $f_i(y) \geq 0$. The theorem holds at any order n, at any number m of constraints, and over any linear ordered field.

Of those three, the form identity is the one not mechanized in general. The abstract lifting identity `bordered_lifting` (same file) is proved at general n: the bordered matrix's quadratic form at $(x, 1)$ equals $x^\top P x + 2b^\top x + c$. Its instantiation to a specific S-procedure assembly is discharged per instance by `ring`, and the capstone theorems carry that instantiation as a hypothesis too.

`certified_global_optimum` (`ClosureOptimum.lean`): given multipliers $\lambda \geq 0$, a witness $\bar{x}$ that minimizes the Lagrangian $f_0 - \sum_i \lambda_i f_i$ over all points, and complementary slackness $\sum_i \lambda_i f_i(\bar{x}) = 0$,

the witness satisfies $f_0(\bar{x}) \le f_0(y)$ at every feasible y — the attainment leg, over any linear ordered field. Its companion `bracket_exact` (same file) adds feasibility of $\bar{x}$ and a supplied lower bound — that $t \le f_0$ on the feasible set — and concludes both $t \le f_0(\bar{x})$ and global optimality of $\bar{x}$. The equality $t^\star = f_0(x^\star)$ that closes the bracket holds because $t^\star$ is formed as the Lagrangian value at $x^\star$, where complementary slackness makes it $f_0(x^\star)$; it is not separately mechanized.

`certified_optimum_pipeline` (`Certificate.lean`) assembles the chain as one theorem. Its hypotheses are an accepted exact factorization $M = L^\top DL$ with pivots ≥ 0, multipliers $\lambda \ge 0$, the form identity, and a feasible Lagrangian-minimizing witness $\bar{x}$ with complementary slackness. It returns three conclusions: t lower-bounds f_0 on the feasible set; $\bar{x}$ is a global optimum; and $t \le f_0(\bar{x})$. From these, $f_0(\bar{x}) = J^\star$ follows because $\bar{x}$ is feasible and minimal. Two fully worked exact-$\mathbb{Q}$ instances, a one-dimensional keep-out and a two-dimensional disk, run through the verdict-level twin `certified_optimum_of_verdict`, which starts from the PSD verdict rather than from a factorization. These are attainment results. The paper's general emitted protocol computes $g(\lambda)$ from the bordered sweep independently of the feasible witness used for the upper bound. It does not require that witness to minimize the Lagrangian. Here $g(\lambda)$ is the Lagrangian infimum. The arrival-value wrapper `general_protocol.py` is an executable implementation of the paper's lemma; no additional Lean refinement theorem is claimed for it.

The elementary elimination steps. `SchurStep.lean` proves `headBordered_pivot_nonneg`: PSD forces a nonnegative head pivot; `schur_complement_psd`: PSD of the bordered matrix and a positive head pivot imply PSD of its Schur complement; and `headBordered_border_zero`: PSD with a zero head pivot forces the border form to vanish. These are the elementary implications used in step (0) of the paper's bordered-sweep proof; they are not a refinement proof for the implementation.

5 The diagonal-dominance leg

`isPSD_of_wdd` (`GershgorinPSD.lean`): symmetry together with row dominance $\sum_{j \ne i} |M_{ij}| \le M_{ii}$ implies PSD at general n over any linear ordered field, and the declaration takes exactly those two hypotheses. This is the Gershgorin leg of the paper's diagonal-dominance lemma.

6 The executable receipts

The paper's measured and instance-level results rest on a set of executable receipts: Python scripts that construct the instances and recompute what the paper reports, most asserting the published values outright. Most certify in exact rational arithmetic through the trusted checkers; the boundary receipts below also run those checkers, using them as the exact reference against a deliberately fixed-precision transcription, and say so where they appear. This section catalogues those receipts by role. The retained reproduction archive, `supplementary-scripts-v26.zip`, records the checks behind the revised main paper. It contains the receipts and their supporting modules; `test_riccati` is retained within the included library snapshot. The archive is part of the revision package; its SHA-256 is `f31aa499f4c61347b8aaf42c39f7b9adc0932e5c07c1e8cf6917da199ec1f65a`. The archive's `README.md` specifies dependencies in `requirements-tested.txt`, the included library snapshot and its `PYTHONPATH`, and the `verify_supplement.py` runner. No additional library checkout is needed. The per-script catalogue is retained in `README-legacy.md`. Throughout the receipts, the bit-size of a rational is `numerator.bit_length()` + `denominator.bit_length()`.

The trusted checkers originate in the Podium repository at `github.com/adi-oltean/podium`; the archive includes their source snapshot and the receipts import it using the documented `PYTHONPATH` (the library import requires NumPy and SciPy). The band checkers are `border_band_psd` and `block_tridiag_psd` in `podium.verify.riccati`; the included snapshot and `test_riccati` are pinned to commit `04e8379`, whose full identifier and source hashes are recorded in the archive manifest. The archived Zenodo release of the library, v0.8.1 (the library's concept DOI 10.5281/zenodo.21302775), predates `podium.verify.riccati`: it carries `podium.verify.bracket` (the dense M-test) and `podium.verify.barrier`, but not the band sweeps.

General-protocol and exact-gap receipts. `general_protocol.py` computes the exact arrival value, including negative finite values, and checks the independent feasible witness. `check_revision.py` checks it against exact row reduction and all principal minors on selected small matrices, exercises singular range failure, and checks equality of every recorded lower endpoint with the pinned library's terminal pivot. Separately, it computes the shifted matrix corner and terminal pivot and calls the PSD predicate for the emitted, loosened and raised lower values. The loosened values pass the PSD test while differing from the emitted endpoint; the raised value fails it. `numerical_proposals.py` runs the three fixed problems in the main paper, retaining the untrusted proposals, rationalization choices and exact verdicts. Its decimal table endpoints are rounded outward from the certified fractions. Each run retains a distinct receipt with status and proposal errors; only a completed run with accepting attempts for all examples supplies a table. `check_proposal_reporting.py` checks missing witnesses, proposal exceptions and invalidation of stale table output. These runs are observations on selected examples, not a coverage or solver-performance result. `check_revision.py`, through `check_findings.exact_gap` and the `exact_gap` entry in `exact-checks.json`, gives matching moment and dual certificates for $d_1^\star = \frac{24319}{1914}$, with true optimum $\frac{212}{13}$ and gap $\frac{89621}{24882}$. `check_cw_height.py` records the positive periodic increment in the accumulated constant and supports the logarithmic assembled height. The archive manifest records source hashes and dependency versions.

The family drivers. Four drivers construct the families of the paper's results table in exact `Fraction` arithmetic, and each verifies its emitted certificate through the exact dense M-test. The trusted checkers refuse floats; the CW state-transition matrix alone is synthesized in floating point and rationalized before any certification.

`shared_state_exact.py` constructs the scalar shared-state family; it pins the results table's scalar row.

`vector_state.py` constructs the vector shared-state family ($d = 1..4$) and pins the table's vector row. Its probe `gap_probe` ($\lambda_k > w_k$ at N=4, d=2) designs the non-PSD candidate certificate of the table's refusal row, on which the checker returns no certificate; the instance itself is benign (`refusal_row_a1_check.py`).

`rpod_cw.py` constructs the CW-coupled keep-out family of the paper's demonstration section and pins the table's inverse-designed CW row. The family's data are mean motion $n_0 = 10^{-3}\,\mathrm{rad/s}$, sampling phase $\tau = n_0\,\Delta t = 1/2$, weights $R = I_4$ and $W_k = w_k I_4$, per-stage radii cycling $\{1/2, 1, 3/2\}$, and STM denominators capped at 10^5. The driver also prints the largest entry of the $d \times d$ block pivots S_k, which grows measurably linearly in the horizon: 780 bits at N=2 to 17889 at N=24; that is reported as data, not derived.

`rpod_cw_freedrift.py` constructs the second CW family, the grid-snapped free-drift keep-out row: states are propagated by the same rationalized Φ and snapped to a fixed decimal grid at every step, so the keep-out radii follow the trajectory instead of being placed on it. It pins that row's columns and re-verifies it at every horizon through three routes: the trusted dense M-test and bordered-M sweep, together with the archive's own block-Riccati sweep in place of the trusted band-H checker.

The separate `freedrift_exact_check.py` receipt propagates the CW state without grid snapping, with radii placed on the propagated states. It checks the exact lower bound through the pinned bordered-PSD predicate and the feasible upper bound through the pinned witness checker, verifies their equality, and refuses a raised lower value. It measures both value and emitted-entry bit-sizes. Its sequence differs from the grid-snapped results-table row; it does not use the dense M-test for this check.

The measurement engines.

`cert_metrics.py` recomputes both bit columns of the results table for the three inverse-designed closing families at the headline sizes, and re-verifies each of those rows through all three trusted routes: the dense M-test, the band-H sweep, and the bordered-M sweep. The fourth closing row, the grid-snapped free-drift family, is re-verified three ways by its own driver rather than here — with the archive's own block-Riccati sweep standing in for the trusted band-H checker.

`freedrift_value_check.py` tests the certified value's size laws on that free-drift family: the divisibility chain, the two denominator bounds, and the pivot bound formed at the bordered and data heights. It does not test the bound formed at the design height, which that family exceeds at every horizon, and says so.

`freedrift_longhorizon_check.py` carries the same family out to $N = 8000$, where the emitted bits rise from 41 to 49: the grid caps the denominator half structurally at 20 bits per coordinate at any horizon, while the numerator tracks the growing state norm.

`ricbits_identity_check.py` runs both CW families' drivers at all 23 horizons and compares their Riccati pivot bit-size columns directly: identical at every one, which is the receipt for the claim that placing the optimum cannot move the PSD structure.

`pivot_gap_allhorizons.py` joins the block-pivot and scalar-pivot bit-size series at every one of the 23 horizons, rather than the five the anatomy receipt happens to cover, and reports their distribution.

`refusal_row_a1_check.py` certifies the *regime* of the results table's refusal row: on that instance the unconstrained minimizer is a strictly feasible exact KKT pair whose Hessian has all-positive exact pivots, so the instance itself is benign and what the row exhibits is the checker refusing a designed non-PSD certificate.

`cw_anatomy.py` recomputes, at all 23 horizons $N = 2, \ldots, 24$, every denominator statistic the paper quotes for the CW-coupled family, together with the closure verdicts and both bit columns. It asserts the divisibility law behind the certified value's denominator, and the value's size inequalities, at every horizon. It measures the entrywise rationalization error of the CW state-transition matrix. Its bordered-sweep section pins the corner-carrying sweep pivot by pivot

against the dense $LDL^\top$ of M at the shortest horizons. The companion paper's bordered-sweep lemma proves the sweep sound; this pinning corroborates that the sweep's recurrences, as the receipt transcribes them, compute the criterion's pivots. The shipped checker itself is pinned at the verdict and terminal-corner-value levels, not pivot by pivot — the same gap that separates the machine-checked acceptance predicate from the shipped code.

`bordered_proof_check.py` exercises the bordered-sweep lemma's two load-bearing steps directly: its verdict claim against the dense M-test, and its *value* claim — that the accumulated corner equals $\gamma - \frac{1}{2}\ell^\top y$ for any y solving $Hy = \frac{1}{2}\ell$ — against a separate exact solve, over 360 adversarial instances (80 of them with indefinite H) the ordinary tests do not reach: singular H with the zero pivot placed first, mid-sweep and last, indefinite H, ℓ exactly in range(H), ℓ purely on $\ker H$, ℓ mixing range and kernel components, and $\ell = 0$. No mismatch on either claim (360/360 verdicts, 160/160 value checks).

The exact instance receipts. Four receipts certify the paper's named instances entirely over $\mathbb{Q}$; the fourth certifies two sibling instances in one run.

`single_keepout_coupling_gap.py` provides a weaker one-sided bound for the paper's duality-gap example. It asserts $J^\star = 212/13$ by exact enumeration of the branch-KKT candidates, certifies through a rational moment point that every level-one certificate is capped at $815/64$, and certifies the level-one gap $\geq 2973/832$. The same receipt discharges the diagnosis: at the unique optimum LICQ pins $\lambda^\star = 24/13$, and $H(\lambda^\star)$ has determinant $-7/4$, so aggregate-PSD fails there.

`two_keepout_stage.py` closes the bracket at $t^\star = 169/320$ on a stage with two active keep-outs whose contacts touch disjoint coordinates; this is one half of the main theorem's multi-keep-out generality. Its published certificate is also the paper's floating-point counterexample object: the same sweep transcribed into IEEE-754 doubles refuses it, the terminal corner coming out -4.44×10^{-16} rather than exactly 0.

`overlap_keepout_stage.py` closes the bracket at $t^\star = 5773/3600$ on a stage whose two active contact gradients share their whole support; this is the other half of the multi-keep-out generality, and the receipt checks that the verifier's normal-equation solve recovers $\lambda^\star$ on exactly that stage.

`hard_dynamics_gap.py` certifies both halves of the paper's hard-dynamics discussion over $\mathbb{Q}$, on d=1 instances with one keep-out and one equality. On the first instance the best constant-multiplier bound is exactly 1 against $J^\star = 3/2$, a certified gap of $1/2$ whose diagnosis is an aggregate-PSD failure. On its sibling the aggregate-PSD analogue holds and the constant-multiplier bracket closes exactly at $t^\star = J^\star = 9/4$.

The pytest pins.

`test_bracket_scaling.py` ships in the archive and pins the receipts above: it executes each of the four exact instance receipts and asserts its published constants, runs `cw_anatomy.py`, and checks `cert_metrics.py`'s results-table rows at the headline sizes.

`test_riccati` ships in the Podium library and pins the band checkers themselves: the band-H sweep on the assembled H and the bordered sweep on the assembled M are checked against the dense reference predicate `is_psd` of `podium.verify.barrier`, on randomized fixed-seed instances plus dedicated singular-H and zero-pivot cases.

The height receipts. These receipts pin the paper's size measures and the separations among its named heights.

`height_measure_check.py` pins the paper's height and bit-size definition: the enumerated least height is attained at the common-denominator form over the lcm of the reduced denominators, per-entry bit-size obeys bit $\leq 2\,\mathrm{ht}$, and a positive integer D read as a rational obeys $\mathrm{bit}(D) = \mathrm{bitlen}(D) + 1$.

`heights_enumerated.py` asserts that on the CW-coupled family the four height-bearing tuples carry four pairwise distinct heights at every one of the 23 horizons, in the strict order $\mathrm{ht}(x^\star) < L < L_M < L_{\mathrm{asm}}$.

`height_inversion.py` exhibits a family admissible under the paper's constructed-family definition on which the assembled height L_{asm} is the *smaller* of the two design-side heights — the receipt behind the statement that the definition forces no ordering between L and L_{asm}.

`height_governance.py` exhibits an admissible family, closing on all three trusted checkers, with $L_M \leq 8$ bits against a design height $L = 257$–258, so that there the per-stage design height bounds every entry of the bordered matrix — the ordering opposite to the CW family's.

`claim311_recheck.py` extends the certified-value measurements past the demonstration window by an exhaustive exact sweep of every horizon $N = 2, \ldots, 6143$: $\mathrm{den}(t^\star)$ stays inside 450–526 bits at every horizon, while $\mathrm{bit}(t^\star)$ keeps the residual $\log N$ drift.

`claim258_const_lm_witness.py` constructs the constant-L_M witness: an admissible closing family with $L_M = 2$ at every horizon whose largest interior pivot's bit-size grows without bound, so no function of L_M alone bounds the sweep's cost.

`attackb_scope.py` constructs the fixed-order witness — at order $n{+}1 = 9$, multipliers of growing denominator raise L_M and the largest interior pivot together, so no function of the order alone bounds the cost either. It also separates the multiplier-recovery solve's operand heights from its formation heights on the demonstration and inverted families.

`attack2_scope.py` **and** `attackb_scope.py` measure the recovery solve's operands at the design height: 5–6 bits against $L = 6$ on the CW family (`attackb_scope.py`), and 259 bits against $L = 258$ on `height_inversion.py`'s inverted family (`attack2_scope.py`).

`claim235_thirdcheck.py` **and** `claim306_attack.py` jointly pin the two witnesses for why the certified value's bit bound carries a horizon term and is not stated against L alone: an admissible closing family with $L_{\mathrm{asm}} = 2$, $\mathrm{ht}(x^\star) = 1$ and $t^\star = N$ exactly, and integer goal weights of growing magnitude taking $\mathrm{den}(t^\star)$ unbounded at fixed $L = 1$.

The class-structure and sweep-scope receipts. These receipts pin structural facts about the class, the scope of the band sweep, and the measured growth boundary of the demonstration family.

`bandwidth_scope.py` pins the scalar bandwidth: the neighbour-only coupling confines every nonzero entry to half-width $b = 2d - 1$, attained whenever some coupling block has a nonzero $(1, d)$ entry, while structured couplings such as $A_k = R_k = I$ sit at half-width exactly d.

`tridiag_structure_scope.py` pins that block tridiagonality of $H(\lambda)$ is forced by the class rather than assumed, with a negative control: the band sweep returns a wrong verdict on a symmetric matrix that lacks the structure.

`curvature_factor_check.py` pins the curvature convention $\nabla^2_{xx}(f_0 - \sum_j \lambda_j f_j) = 2H(\lambda)$ under the paper's no-$\frac{1}{2}$ convention, and enumerates which downstream uses of H are invariant under positive rescaling.

`band_lemma_scope.py` extends the band-$LDL^\top$ soundness and zero-pivot checks to general bandwidth, including even b, which is the generality at which the paper states its band lemma.

`band_opcount.py` instruments the shipped band sweep and pins its $O(nb^2)$ count for $b \geq 1$, consistent with the papers' $O(n(b+1)^2) = O(Nd^3)$ bound at trajectory bandwidth. It is the band sweep's count that is measured; the bordered sweep's count is read off its loop bounds, as the paper's bordered-extension remark states.

`wdd_row_reduction_check.py` pins the identity-coupled row shapes of the paper's diagonal-dominance lemma and the reduction of WDD to the local test $w_k \geq \lambda_k$; the checks cover the singular boundary, including the case where the guarded pivot recursion is silent but the local test is not.

`riccati_margin.py` demonstrates that WDD is sufficient but not necessary for the PSD verdict: on the constant-diagonal scalar chain tridiag$(-1, c, -1)$ — the identity-coupled row shapes with an excess one larger on the two boundary rows than in the interior, a mixed-sign pattern in the paper's sense — PSD persists for $N \geq 3$ below the interior-row WDD threshold $c = 2$, down to $2\cos(\pi/(N+1))$. On the paper's own nonnegative-excess ray the two thresholds coincide, which is the necessity the paper states there.

`ya_scope.py` pins the boundary of the demonstration family's growth behaviour: its mild pivot growth and its compact recomputed value are properties of that family. Swapping the shared CW matrix for the per-stage Yamanaka–Ankersen transition matrix, with everything else unchanged, reproduces the CW pivot sequence bit for bit at $e = 0$. At the probed eccentricities $e \in \{0.1, 0.3, 0.5\}$, with the stage matrices rationalized to the CW family's realized per-entry fidelity, the pivots grow at 1037–1049 bits per stage against CW's ≈ 778, and bit$(t^\star)$ grows at ≈ 1000 bits per stage, where on the CW family it shows no per-stage growth: 931–1087 bits across the receipt's sampled horizons $N \in \{2, 4, 8, 16, 24, 48\}$, non-monotone. The penalty is already present at $e = 10^{-4}$, so it does not track eccentricity magnitude. That bit$(t^\star)$ grows at all here is a property of the sharing; the pivot-slope excess is not attributed here.

The boundary receipts. Five receipts pin the fixed-precision boundary analysis of the paper's related-work discussion.

Positive-margin cases. Pivots away from the semidefinite boundary may round, with their positive-width enclosures absorbed by their positive margins. The receipts are `claim269_recheck.py`, `claim305_recheck.py`, and `claim305_deepchain.py`.

`claim305_recheck2.py` pins the exactly-computed-zero-pivot scope of the Cholesky clause: LAPACK's `dpotrf` declines the factorization of the singular H with rows $(1, -1)$ and $(-1, 1)$. That an $LDL^\top$ and the symmetric eigensolver are exact there is pinned by `attack3_scope.py` and `claim269_recheck.py`.

`attack3_scope.py` is the receipt named in the simplified paper's closure appendix. It checks that an IEEE-754 double transcription of the sweep refuses `two_keepout_stage.py`'s certificate at a negative terminal corner (reported as -4.44×10^{-16}), and checks the zero-pivot branch

on which the bordered and band-H pivot sequences diverge. The assertion tests negativity and refusal, not equality to that printed floating-point value.

7 Mechanization of the published note's theorems

The exact S-procedure bracket and the rational $LDL^\top$ PSD test for single primitives are established in the published Podium technical note, *Exact-Rational Certificates in Podium: Constructions, Proofs, and Prior Art* [3]. Its soundness core is mechanized in `PodiumNote.lean`, which sits in a `note/` subdirectory beside the nineteen top-level files. Its manifest SHA-256 is `36e448642266b41f60eb232720aa665699fb35add3176001cc6ac390c0da95dd`. The snapshot compilation described in §2 checked this file along with the nineteen top-level files, under the same toolchain pin, and recorded the axiom audit for `note_thm_sound`, `note_thm_multi` and `note_lem_schur`. The declarations are stated over any linear ordered field (this statement concerns the cited declarations). They mechanize the $m = 1$ instance of the general-m minorant inequality in the note's Theorem 1, the fixed-multiplier inequality used by its Theorem 4 and the accompanying pointwise bracket inequality, and the scalar case of its Schur-complement lemma.

`note_thm_sound` (the note's Theorem 1, the lower bound, at one constraint): if $\lambda \geq 0$ and the minorant condition $t \leq f_0(x) - \lambda f_1(x)$ holds at every point x, then $t \leq f_0(x)$ at every x with $f_1(x) \geq 0$. Passing to the infimum to obtain $t \leq J^\star$ is an on-paper step. The minorant condition is where $M(\lambda, t) \succeq 0$ enters: the published note derives it from PSD of the bordered matrix by the lifting expansion, and the declaration takes the minorant itself as its hypothesis. Within the corpus, the same step is carried at general order by `general_sprocedure_sound`'s form-identity hypothesis (§4).

`note_thm_multi` (the note's Theorem 4, Certified bounds on suboptimality and duality gap, at any number m of constraints): if every $\lambda_i \geq 0$ and $t \leq f_0(x) - \sum_i \lambda_i f_i(x)$ at every point, then $t \leq f_0(x)$ at every x with all $f_i(x) \geq 0$. This is the fixed-minorant inequality that Theorem 4 takes from the general-m Theorem 1. The declaration is stated at one fixed (λ, t); taking the supremum defines the S-procedure dual value $d^\star$ (the $d_1^\star$ of §6). Identifying it with the Shor primal relaxation value requires SDP strong duality. The supremum step, that identification, and strict gap examples are outside this declaration.

`note_bracket` (the bracket and suboptimality part of the same theorem): a lower bound $t \leq f_0$ over a feasible set, supplied as a hypothesis, together with a feasible witness $\bar{x}$, yields $t \leq f_0(\bar{x})$ and, at every feasible x, $f_0(\bar{x}) - (f_0(\bar{x}) - t) \leq f_0(x)$ — the form in which the width $f_0(\bar{x}) - t$ of the bracket $[t, f_0(\bar{x})]$ bounds the suboptimality of $\bar{x}$. Feasibility enters as an abstract predicate, so the statement is independent of the constraint form. It quantifies over feasible points; the passage to $J^\star = \inf f_0$ remains on paper. Neither $d^\star$ nor the theorem's duality-gap bound appears in this declaration.

`note_lem_schur` (the note's Schur-complement lemma, scalar case): for $a > 0$, the bordered matrix $M = \left[\begin{smallmatrix} a & b/2 \\ b/2 & c-t \end{smallmatrix}\right]$ is PSD in the quantifier sense $\forall v,\ 0 \leq v^\top M v$ if and only if $t \leq c - b^2/(4a)$; the right-hand side is the dual value $\inf_x(ax^2 + bx + c)$. The equivalence is mechanized in both directions. The published lemma is stated for a matrix block $A(\lambda) \succ 0$ with $g(\lambda) = c - \frac{1}{4} b^\top A(\lambda)^{-1} b$; the declaration is its 1×1 instance.

Recovery-rate declarations. `RecoveryRate.lean` proves bounds over $\mathbb{Q}$ for supplied exact quadratic or absolute-value closed forms of g and a supplied multiplier-error bound. It does not establish that a general QCQP dual has those closed forms, or mechanize the analytic asymptotics of the note's recovery theorems.

8 Scope: what the theorems take as hypotheses

The shipped sweeps are not the mechanized object: the corpus proves the acceptance predicate's equivalence with positive semidefiniteness. The claim that the concrete band elimination in `podium.verify.riccati` computes such a factorization is a refinement hypothesis, taken rather than proved. The same holds for the exact dense PSD predicate `is_psd` of `podium.verify.barrier` — the predicate `test_riccati` pins the band elimination against, and on which the dense M-test of `podium.verify.bracket` rests — through which each of the four family drivers checks its emitted certificate.

On the sweep side, three things sat outside the mechanized perimeter: the border-carrying sweep; the band-restriction equality (band pivots equal dense pivots); and the operation count. They are not all in the same epistemic state, and the distinction is worth keeping visible — the reader's ledger has three columns: mechanized, proved on paper, and asserted.

As of 2026-08-26 the bordered sweep is *split across all three*. `BorderedCriterion.lean` machine-checks the *dense criterion* the sweep is proved against — for the bordered $[[\gamma, b^\top], [b, H]]$ with symmetric H: acceptance data (H PSD, a solution of $Hy = b$, and $b \cdot y \leq \gamma$) implies PSD (`borderedCriterion_sound`); PSD forces H PSD, the border orthogonal to $\ker H$ (the range condition, kernel side) and the corner bound at any solution (`borderedCriterion_head_psd`, `borderedCriterion_ker`, `borderedCriterion_value`); the corner value is independent of the solution chosen (`borderedValue_welldef`); packaged as an equivalence under solvability (`borderedCriterion_iff`). General n, any linear ordered field, no `sorry`, the three standard axioms only. The mechanized matrix carries its border at the *head*, where the shipped sweep's M carries it last; the identification is a simultaneous row/column permutation, under which PSD is invariant — a standard fact used here, whose instantiation to this permutation is not mechanized (the file states the invariance in prose and declares no such lemma). What remains *proved on paper only* is the bordered-sweep lemma's locality half — that the *banded* algorithm with its restricted index ranges computes this criterion (its steps (2) and (3)) — together with the algorithm-to-criterion bridge inside its steps (1), (4) and (5), the congruence and factorization arguments identifying the sweep's recurrences with the criterion's data; both concern the algorithm rather than the predicate. Also on paper is the finite-dimensional implication from border orthogonality to $\ker H$ to solvability of $Hy = b$; the formal equivalence assumes solvability. The band-restriction equality sits in the same column. The operation count of step (7) is in that same column, *proved on paper*: a bound read off the shipped loop bounds, and measured by no test. What is *asserted* rather than proved is the border's position — the permutation identifying the mechanized head layout with the sweep's last-border layout. `StructuralPSD.lean`'s `isPSD_congr` machine-checks that congruence preserves PSD at general order, but nothing instantiates it to this permutation, and `BorderedCriterion.lean` states the invariance only in prose.

The form identity is discharged per instance: `general_sprocedure_sound`'s third hypothesis and the capstone's matching hypothesis are proved by `ring` for each concrete assembly, never once in general.

Lagrangian minimality is supplied from outside the corpus: `certified_global_optimum` takes it as a hypothesis, and the quadratic-convexity step that produces it in the paper's proof is not mechanized.

The bridge is compiled separately, and so is everything else: no file in the corpus imports another, and each re-declares the predicates it needs. Ten top-level files declare their own `IsPSD` under the same unnamespaced name and with the same formula — among them `PosSemidefBridge.lean`, `LDLTSound.lean`, `SchurStep.lean`, `GershgorinPSD.lean` and `BorderedCriterion.lean`. So no compiled artifact links the bridge, the acceptance predicate, the Gershgorin leg or the bordered

criterion to one another: the formulas are definitionally identical over $\mathbb{Q}$, and every match is read off by inspection rather than by the compiler.

The second leg of the paper's diagonal-dominance lemma is the Schur-complement preservation of weak diagonal dominance, resting on Carlson–Markham [1] together with the limiting argument in that proof. That leg is outside the corpus; only the Gershgorin leg is machine-checked.

References

[1] David Carlson and Thomas L. Markham. Schur complements of diagonally dominant matrices. *Czechoslovak Mathematical Journal*, 29(2):246–251, 1979.

[2] Adi Oltean. Auditable exact lower bounds for trajectory QCQPs via a rational Riccati sweep, 2026. Companion paper, Zenodo, `https://doi.org/10.5281/zenodo.22035914`; cross-references here use manuscript version 27.

[3] Adi Oltean. Exact-rational certificates in podium: Constructions, proofs, and prior art, 2026. Technical note, Zenodo, `https://doi.org/10.5281/zenodo.21247380`; cross-references here use the September 8, 2026 manuscript revision.